

Teach Module 13 README

Module 13: SOAP API Design with Java

This lesson uses the course document only to identify the Module 13 topic. The teaching content below is written independently and focuses on understanding SOAP API design before implementation.

Learning Goals

By the end of this module, you should be able to explain:

- What SOAP is and why enterprise systems still use it.
- How SOAP differs from REST.
- What a SOAP envelope, header, body, and fault are.
- What a WSDL contract describes.
- Why SOAP services are often designed contract-first.
- How to design operations, request messages, response messages, faults, and versions.

1. What SOAP Is

SOAP stands for Simple Object Access Protocol. Despite the name, SOAP is usually more formal than REST. It is common in banks, insurance, healthcare, government systems, payment systems, and legacy enterprise integrations because it provides a strict contract.

The big idea:

SOAP APIs are contract-first services where the client and server agree on an exact XML message format before implementation.

Think of SOAP like a legal form. REST often says, "Send JSON to this URL." SOAP says, "Here is the exact XML structure, operation name, request type, response type, fault type, namespace, and transport rule."

2. Why SOAP Exists

SOAP was designed for enterprise systems that need:

- Strict contracts
- Predictable message structure
- Platform independence
- Formal error handling
- Strong tooling support
- Security and reliability standards
- Compatibility across Java, .NET, mainframes, and legacy systems

Example:

A bank exposes a `GetAccountBalance` service. Many clients may consume it: Java apps, .NET apps, batch jobs, partner systems, and older middleware.

SOAP helps because everyone follows the same WSDL contract.

3. SOAP vs REST

REST is usually resource-oriented:

```
GET /accounts/123/balance
```

SOAP is usually operation-oriented:

```
GetAccountBalance
```

REST commonly sends JSON:

```
{  
  "accountId": "123"  
}
```

SOAP sends XML inside a formal envelope:

```
<soapenv:Envelope>  
  <soapenv:Header/>  
  <soapenv:Body>  
    <GetAccountBalanceRequest>  
      <accountId>123</accountId>  
    </GetAccountBalanceRequest>  
  </soapenv:Body>  
</soapenv:Envelope>
```

REST feels like browsing resources. SOAP feels like calling a remote business operation with a strict document attached.

4. SOAP Message Structure

Every SOAP message has an envelope.

```
Envelope  
  Header  
  Body
```

The header contains metadata, such as:

- Authentication token
- Correlation ID
- Transaction ID
- Locale
- Routing information
- Security signature

The body contains the actual request or response.

Example:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Header>
    <correlationId>abc-123</correlationId>
  </soapenv:Header>

  <soapenv:Body>
    <GetCustomerRequest>
      <customerId>42</customerId>
    </GetCustomerRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

SOAP does not casually send data. It wraps data in a standard message format.

5. WSDL: The SOAP Contract

WSDL means Web Services Description Language.

A WSDL file tells clients:

- What operations exist
- What request message each operation expects
- What response message each operation returns
- What faults can happen
- Where the service is located
- Which protocol and binding are used

Example service contract idea:

Operation: GetAccountBalance

Input:

accountId: string

Output:

accountId: string

currentBalance: decimal

currency: string

Faults:

AccountNotFoundFault

UnauthorizedAccessFault

In SOAP design, you usually do not start by writing Java controllers. You first design the contract.

That is called contract-first design.

6. Contract-First Design

In many REST projects, developers start with code:

```
@GetMapping("/customers/{id}")
public Customer getCustomer(...) {
    // implementation
}
```

In SOAP, a strong approach is:

1. Define the XML schema.
2. Define the WSDL.
3. Generate Java classes from the contract.
4. Implement the service behavior.

This keeps both sides honest:

- The client knows exactly what to send.
- The server knows exactly what to receive.
- The API contract becomes the source of truth.

7. SOAP Operations

SOAP operations are named business actions.

Examples:

```
CreateCustomer
GetCustomer
UpdateCustomerAddress
SubmitLoanApplication
CalculatePremium
ProcessPayment
```

Good SOAP operations are business-focused.

Good names:

```
GetAccountBalance
SubmitPayment
ValidateCustomerIdentity
```

Weak names:

```
DoStuff
ProcessData
HandleRequest
```

SOAP APIs are usually used for important enterprise workflows, so operation names should be clear and stable.

8. SOAP Faults

SOAP has a formal way to return errors called a fault.

A SOAP fault usually tells the client:

- What went wrong
- Whether the error came from the client or server
- A fault code
- A fault message
- Application-specific details

Example:

```
<soapenv:Fault>
  <faultcode>soapenv:Client</faultcode>
  <faultstring>Invalid account ID</faultstring>
  <detail>
    <errorCode>ACCOUNT_ID_INVALID</errorCode>
  </detail>
</soapenv:Fault>
```

A business error is not always a system crash.

Examples:

```
Account not found
Insufficient funds
Customer not eligible
Invalid policy number
```

These should be designed as predictable faults, not random exceptions.

9. Versioning SOAP APIs

SOAP contracts are strict. That is useful, but it means changes must be careful.

Breaking changes include:

- Renaming an XML element
- Removing a required field
- Changing a field type
- Changing an operation name
- Changing namespaces unexpectedly

Safer changes include:

- Adding optional fields
- Adding a new operation
- Creating a new versioned namespace

Example namespaces:

```
http://example.com/banking/v1
http://example.com/banking/v2
```

A common SOAP versioning strategy is to keep old contracts alive while introducing a new version.

10. Java's Role in SOAP

In Java, SOAP is commonly implemented using tools and frameworks such as:

- Spring Web Services
- JAX-WS
- JAXB for XML binding
- WSDL/XSD-based class generation

For Module 13, the main focus is design, not implementation.

The design flow is:

```
Business operation
XML schema
WSDL contract
SOAP request/response
Java service implementation
```

Implementation comes later.

Mini Design Example

Suppose we need a SOAP API for checking loan eligibility.

Operation:

```
CheckLoanEligibility
```

Request:

```
customerId
annualIncome
creditScore
requestedAmount
```

Response:

```
eligible
approvedAmount
reasonCode
```

Faults:

```
InvalidCustomerFault
CreditServiceUnavailableFault
InvalidLoanAmountFault
```

Design questions:

- Is `creditScore` required?
- What happens if the customer ID is invalid?
- Should unavailable credit data be a fault or a response reason?
- Should money use decimal instead of float?
- Should currency be included?
- Should this operation be versioned under `v1` ?

This is SOAP API design thinking. You are not just asking, "What Java method should I write?" You are asking, "What contract can many systems safely depend on for years?"

Practice Exercises

Exercise 1: Identify SOAP Use Cases

For each scenario, decide whether SOAP is a good fit or REST is better:

1. Bank balance inquiry between two enterprise systems
2. Public weather API for mobile apps
3. Insurance claim submission to a government system
4. Internal product catalog lookup
5. Payment settlement between banks

For each one, write:

SOAP or REST:
Reason:
Need for strict contract:
Need for formal faults:

Exercise 2: Design a SOAP Operation

Design a SOAP operation called:

`GetCustomerProfile`

Define:

Operation name:
Request fields:
Response fields:
Possible faults:
Header values:
Namespace/version:

Example fault ideas:

`CustomerNotFoundFault`
`UnauthorizedAccessFault`
`InvalidCustomerIdFault`

Exercise 3: Create a SOAP Request Envelope

Write a SOAP XML request for:

```
Operation: GetAccountBalance
```

Required fields:

```
accountId  
customerId
```

Header should include:

```
correlationId  
clientId
```

Exercise 4: Create a SOAP Response Envelope

Write a SOAP XML response for:

```
GetAccountBalanceResponse
```

Fields:

```
accountId  
availableBalance  
currentBalance  
currency  
asOfDateTime
```

Think carefully about data types:

```
decimal  
string  
dateTime
```

Exercise 5: Design SOAP Faults

For the `TransferFunds` operation, design faults for these cases:

1. Source account does not exist
2. Destination account does not exist
3. Insufficient funds
4. Invalid transfer amount
5. System temporarily unavailable

For each fault, define:

Fault name:
Fault code:
Fault message:
Detail fields:
Client or server fault:

Exercise 6: Contract-First Thinking

Design a SOAP service named:

LoanEligibilityService

It should have three operations:

CheckEligibility
SubmitLoanApplication
GetApplicationStatus

For each operation, define:

Input message:
Output message:
Faults:
Business purpose:

Exercise 7: Versioning Challenge

You already have this response contract:

GetCustomerProfileResponse
- customerId
- firstName
- lastName
- email

Now the business wants to add:

phoneNumber
preferredLanguage
marketingOptIn

Answer:

Which changes are safe?
Which changes could break clients?
Would you create v2?
What would the namespace be?

Example namespace:

```
http://example.com/customer/v2
```

Exercise 8: WSDL Reading Practice

Without writing a full WSDL, describe what a WSDL for `OrderStatusService` should contain:

```
Service name:
Operations:
Input messages:
Output messages:
Fault messages:
Endpoint URL:
Binding style:
Namespace:
```

Exercise 9: API Design Review

Review this weak SOAP operation:

```
Operation: ProcessData
Request:
  data: string

Response:
  result: string

Fault:
  ErrorFault
```

Improve it. Rename the operation, split vague fields, and create better faults.

Example direction:

```
Operation: ValidateCustomerIdentity

Request:
  customerId
  dateOfBirth
  lastFourSsn

Response:
  verificationStatus
  reasonCode

Faults:
  InvalidCustomerIdFault
  IdentityServiceUnavailableFault
```

Exercise 10: Full Mini Project

Design a complete SOAP API for:

PaymentService

Include:

Operations:

InitiatePayment
GetPaymentStatus
CancelPayment

Headers:

correlationId
clientId
authToken

Faults:

PaymentNotFoundFault
DuplicatePaymentFault
InvalidPaymentAmountFault
UnauthorizedClientFault
PaymentSystemUnavailableFault

Version:

v1 namespace

Deliverables:

1. Service overview
2. Operation list
3. Request/response fields
4. Fault definitions
5. Sample SOAP request
6. Sample SOAP response
7. Versioning strategy

Recommended practice order:

1. Exercise 2
2. Exercise 3
3. Exercise 4
4. Exercise 5
5. Exercise 10

Module 13 Lab: Design a SOAP API Contract

Lab Goal

Design a SOAP API for a simple enterprise banking service. You do not need to implement it in Java yet. The focus is API design, XML message structure, faults, and versioning.

Scenario

A bank needs a SOAP service that allows trusted partner systems to:

1. Get account balance
2. Transfer funds
3. Check transfer status

Service name:

BankingTransactionService

Namespace:

http://example.com/banking/v1

Part 1: Define the Service

Write a short service overview:

Service Name:
Business Purpose:
Consumers:
Version:
Namespace:

Example:

Service Name: BankingTransactionService
Business Purpose: Allows partner systems to check balances and perform account transfers.
Consumers: Internal banking apps and approved partner systems.
Version: v1
Namespace: http://example.com/banking/v1

Part 2: Define Operations

Define these three SOAP operations:

GetAccountBalance
TransferFunds
GetTransferStatus

For each operation, write:

Operation Name:
Purpose:

Request Fields:
Response Fields:
Possible Faults:

Part 3: Design Request and Response Models

Use this format.

For `GetAccountBalanceRequest` :

```
accountId: string, required
customerId: string, required
```

For `GetAccountBalanceResponse` :

```
accountId: string
availableBalance: decimal
currentBalance: decimal
currency: string
asOfDateTime: dateTime
```

Now design the request and response models for:

```
TransferFunds
GetTransferStatus
```

Suggested fields for `TransferFundsRequest` :

```
sourceAccountId
destinationAccountId
amount
currency
transferMemo
```

Suggested fields for `TransferFundsResponse` :

```
transferId
status
submittedAt
```

Suggested fields for `GetTransferStatusRequest` :

```
transferId
```

Suggested fields for `GetTransferStatusResponse` :

```
transferId
status
```

```
sourceAccountId
destinationAccountId
amount
currency
lastUpdated
```

Part 4: Define SOAP Headers

Every request should include these SOAP headers:

```
correlationId: string
clientId: string
authToken: string
requestTimestamp: dateTime
```

Write a short explanation of each header.

Example:

```
correlationId: Used to trace a request across systems.
```

Part 5: Define Faults

Create fault definitions for:

```
InvalidAccountFault
AccountNotFoundFault
InsufficientFundsFault
UnauthorizedClientFault
DuplicateTransferFault
BankingSystemUnavailableFault
```

For each fault, write:

```
Fault Name:
When It Happens:
Client or Server Fault:
Fault Message:
Detail Fields:
```

Example:

```
Fault Name: InsufficientFundsFault
When It Happens: Source account does not have enough available balance.
Client or Server Fault: Client
Fault Message: Source account has insufficient available funds.
Detail Fields:
    sourceAccountId
    availableBalance
```

```
requestedAmount
currency
```

Part 6: Write a Sample SOAP Request

Create a SOAP request for:

```
GetAccountBalance
```

Use this starter:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:bank="http://example.com/banking/v1">
  <soapenv:Header>
    <bank:correlationId>REQ-1001</bank:correlationId>
    <bank:clientId>PARTNER-APP-01</bank:clientId>
    <bank:authToken>sample-token</bank:authToken>
    <bank:requestTimestamp>2026-08-01T10:30:00Z</bank:requestTimestamp>
  </soapenv:Header>

  <soapenv:Body>
    <bank:GetAccountBalanceRequest>
      <bank:accountId>ACC-12345</bank:accountId>
      <bank:customerId>CUST-98765</bank:customerId>
    </bank:GetAccountBalanceRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

Part 7: Write a Sample SOAP Response

Create a SOAP response for:

```
GetAccountBalanceResponse
```

Use this starter:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:bank="http://example.com/banking/v1">
  <soapenv:Header>
    <bank:correlationId>REQ-1001</bank:correlationId>
  </soapenv:Header>

  <soapenv:Body>
    <bank:GetAccountBalanceResponse>
      <bank:accountId>ACC-12345</bank:accountId>
      <bank:availableBalance>1250.75</bank:availableBalance>
      <bank:currentBalance>1300.75</bank:currentBalance>
      <bank:currency>USD</bank:currency>
      <bank:asOfDateTime>2026-08-01T10:30:02Z</bank:asOfDateTime>
    </bank:GetAccountBalanceResponse>
  </soapenv:Body>
</soapenv:Envelope>
```

```
</soapenv:Body>
</soapenv:Envelope>
```

Part 8: Write a SOAP Fault Example

Create a SOAP fault for invalid account ID.

Starter:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/"
  xmlns:bank="http://example.com/banking/v1">
  <soapenv:Body>
    <soapenv:Fault>
      <faultcode>soapenv:Client</faultcode>
      <faultstring>Invalid account identifier.</faultstring>
      <detail>
        <bank:InvalidAccountFault>
          <bank:errorCode>INVALID_ACCOUNT_ID</bank:errorCode>
          <bank:message>Account ID format is invalid.</bank:message>
          <bank:fieldName>accountId</bank:fieldName>
        </bank:InvalidAccountFault>
      </detail>
    </soapenv:Fault>
  </soapenv:Body>
</soapenv:Envelope>
```

Part 9: Versioning Questions

Answer these:

1. What changes can be added safely to v1?
2. What changes would break existing clients?
3. When should the service move to v2?
4. What namespace would v2 use?

Example v2 namespace:

```
http://example.com/banking/v2
```

Part 10: Final Lab Deliverable

Create one document or markdown file containing:

1. Service overview
2. Operation definitions
3. Request and response field definitions
4. Header definitions
5. Fault definitions
6. Sample SOAP request
7. Sample SOAP response

- 8. Sample SOAP fault
- 9. Versioning answers

Success Criteria

You are done when you can clearly explain:

- What the service does
- What each operation expects
- What each operation returns
- What can go wrong
- How faults are represented
- How versioning is handled
- Why this is contract-first design